

Pedra, Papel e Tesoura: *definindo o placar*

Última etapa do jogo: um **placar** com vitórias, derrotas, empates e rodadas, um **histórico** das últimas jogadas e um botão para **zerar**. No caminho, um detalhe do Streamlit que pega muita gente: por que o placar "atrasa uma rodada" e como o `st.rerun()` resolve.

`st.metric`

Placar em `session_state`

Histórico

`st.expander`

Ordem de execução

`st.rerun()`

Formação Claude Code 2026 · Baseado em cursor.com/docs (Agent, Debug Mode, Browser) e docs.streamlit.io (`st.metric`, `st.expander`, Session State, `st.rerun`), consultados em outubro de 2026 (Streamlit 1.65) · Código de referência testado em [projeto/](#) · Telas: print real

Da versão 2 *para a versão 3*

ESCOPO DA V3

- ✓ Placar no topo: Você · Computador · Empates · Rodadas
- ✓ Atualiza na hora, a cada jogada
- ✓ Histórico das últimas 5 rodadas, num painel que abre e fecha
- ✓ Botão Zerar **placar**
- ✓ Imagens e regra como na v2

DADOS NOVOS

Dois itens na sessão: o **placar**, um dicionário com três contadores, e o **histórico**, uma lista de rodadas com a mais recente primeiro.

```
placar = {"vitoria": 4, "derrota": 2, "empate": 1}
historico = [("Pedra", "Tesoura", "vitoria"), ...]
```

As chaves do placar são as mesmas palavras que `resultado()` devolve. Somar um ponto vira uma linha: `placar[final] += 1`.

O pedido *da versão 3*

Adicione um placar ao jogo:

- `st.session_state.placar` com vitórias, derrotas e empates, usando as mesmas palavras que `resultado()` devolve;
- mostre no topo, com `st.metric`: Você, Computador, Empates e Rodadas;
- guarde as rodadas num histórico e mostre as 5 últimas dentro de um `st.expander`;
- botão "Zerar placar" que limpa placar e histórico.

O placar precisa mostrar o valor certo logo depois de cada jogada. Não mude o `jogo.py`.

"Logo depois de cada jogada" parece óbvio, mas é exatamente onde a primeira versão costuma errar. Dizer isso no pedido vira critério de teste (seção 05).

Peça ao agente para explicar **a ordem** em que o script desenha placar, botões e rodada. Entender a ordem é entender o Streamlit.

Placar *no topo*

```
MENSAGEM = {"vitoria": "Você venceu! 🏆", "derrota": "O computador venceu.", "empate": "Empate!"}

if "placar" not in st.session_state:
    st.session_state.placar = {"vitoria": 0, "derrota": 0, "empate": 0}
    st.session_state.historico = [] # rodadas mais recentes primeiro

placar = st.session_state.placar

# Placar no topo
c1, c2, c3, c4 = st.columns(4)
c1.metric("Você", placar["vitoria"])
c2.metric("Computador", placar["derrota"])
c3.metric("Empates", placar["empate"])
c4.metric("Rodadas", sum(placar.values()))

colunas = st.columns(3)
for coluna, opcao in zip(colunas, OPCOES):
    coluna.image(str(IMAGEM[opcao]), width="stretch")
    if coluna.button(opcao, key=f"jogar_{opcao}", width="stretch"):
        computador = jogada_do_computador()
        final = resultado(opcao, computador)
        placar[final] += 1
        st.session_state.historico.insert(0, (opcao, computador, final))
        st.rerun()
```

- ✓ O placar é criado **uma vez**, na primeira execução
- ✓ `st.metric` mostra cada número com rótulo
- ✓ Rodadas = soma dos três contadores
- ✓ No clique: sorteia, decide, soma o ponto, guarda no histórico
- ✓ `insert(0, ...)` põe a rodada nova no começo da lista

Repare no `st.rerun()` no fim do clique. A próxima seção mostra o que acontece sem ele.

O placar que *atrasa uma rodada*

O Streamlit roda o script **de cima para baixo** a cada clique. O placar é desenhado **antes** dos botões:

- 1 Você clica em Pedra. O script recomeça.
- 2 Desenha o placar com os valores **antigos**.
- 3 Chega no botão, vê o clique e soma o ponto. Mas o placar já foi desenhado.
- 4 Sem `st.rerun()`, a tela mostra o placar **uma rodada atrasado**.

DUAS SOLUÇÕES

A) Depois de mudar o placar, chamar `st.rerun()`: o script roda de novo, e agora o placar já começa com o valor novo. É a do código de referência.

B) Usar `on_click` no botão: a função roda **antes** do script, então o placar já é desenhado atualizado.

Se o agente entregar sem nenhuma das duas, o teste da seção 05 pega. Descreva o sintoma ("o placar só atualiza no clique seguinte") e peça para explicar a causa.

Testando *o placar*

```
def test_placar():
    at = AppTest.from_file("app.py").run()
    for computador in ["Tesoura", "Papel", "Pedra", "Tesoura"]:
        with mock.patch("random.choice", return_value=computador):
            at.button(key="jogar_Pedra").click().run()
    assert at.session_state.placar == {
        "vitoria": 2, "derrota": 1, "empate": 1}
    # o que aparece na tela, logo após a última jogada
    assert [m.value for m in at.metric] == ["2", "1", "1", "4"]
```

O código de referência passou: Pedra contra Tesoura, Papel, Pedra e Tesoura dá 2 vitórias, 1 derrota e 1 empate, e o **placar na tela** mostra 2 · 1 · 1 · 4 logo depois da última jogada.

- ✓ Confere o **estado** (session_state) e a **tela** (st.metric)
- ✓ A segunda conferência é a que pega o "atraso de uma rodada"
- ✓ Um teste a mais: **Zerar placar** volta tudo a zero e esconde o histórico

Testar só o estado não basta: ele estava certo também na versão com o atraso. O usuário vê a tela.

Histórico *e zerar*

```
historico = st.session_state.historico
if historico:
    jogador, computador, final = historico[0]
    st.divider()
    lado_a, meio, lado_b = st.columns([2, 1, 2], vertical_alignment="center")
    lado_a.image(str(IMAGE[jogador]), caption=f"Você: {jogador}", width=160)
    meio.markdown("<h2 style='text-align:center'>x</h2>", unsafe_allow_html=True)
    lado_b.image(str(IMAGE[computador]), caption=f"Computador: {computador}", width=160)
    {"vitoria": st.success, "derrota": st.error, "empate": st.info}[final](MENSAGEM[final])

    with st.expander("Últimas rodadas"):
        for jogador, computador, final in historico[:5]:
            st.write(f"{jogador} × {computador} → {MENSAGEM[final]}")

if st.button("Zerar placar"):
    del st.session_state.placar
    del st.session_state.historico
    st.rerun()
```

- ✓ A rodada mostrada é sempre `historico[0]`, a mais recente
- ✓ O dicionário escolhe a função de mensagem: `st.success`, `st.error` ou `st.info`
- ✓ `st.expander` esconde o histórico até você abrir
- ✓ Zerar **apaga** as duas chaves; na próxima execução, o placar é recriado com zeros

A versão 3 *funcionando*

Pedra, Papel e Tesoura

Você	Computador	Empates	Rodadas
4	2	1	7



Placar no topo · print real



Você: Pedra

Computador: Tesoura

Você venceu! 🎉

Últimas rodadas

Pedra x Tesoura → Você venceu! 🎉

Papel x Tesoura → O computador venceu.

Tesoura x Tesoura → Empate!

Papel x Pedra → Você venceu! 🎉

Pedra x Tesoura → Você venceu! 🎉

Última rodada, histórico aberto e Zerar placar · print real

Ideias para *treinar sozinho*

MELHOR DE 5

Quem chegar a 3 vitórias primeiro ganha a partida; mostre um aviso de campeão e ofereça uma nova partida.

SEQUÊNCIA

Mostre a maior sequência de vitórias e a sequência atual.

PLACAR SALVO

Guarde o placar num SQLite, como no ToDo List (aula 12), para ele sobreviver ao F5.

COMPUTADOR ESPERTO

Em vez de sortear, o computador joga contra a sua jogada mais frequente. A regra continua em `jogo.py`, testável.

Para cada ideia: um pedido pequeno, um teste que prove que funcionou e um commit. É o ritmo dos três projetos até aqui.

O jogo *em três aulas*

VERSÃO	O QUE ENTROU	LIÇÃO
v1 · aula 13	Botões, sorteio e resultado	Regra separada da tela; testar as 9 combinações; sorte controlada com mock
v2 · aula 14	Imagens e rodada lado a lado	Assets com <code>Path(__file__)</code> ; guardar a rodada na sessão; ajustar pelo print
v3 · aula 15	Placar, histórico e zerar	Ordem de execução do Streamlit; <code>st.rerun()</code> ; testar a tela, não só o estado

```
git add app.py
git commit -m "Pedra, Papel e Tesoura: placar e histórico"
```

Checklist: placar certo logo após cada jogada, histórico com as 5 últimas, zerar funciona, testes das aulas 13 e 14 passando.

Teste o que *ficou*

1. Por que o placar poderia aparecer "uma rodada atrasado"?

A) O sorteio é lento

B) Ele é desenhado antes de o clique somar o ponto

C) Bug do st.metric

D) Falta banco de dados

B. O script roda de cima para baixo.

2. Qual destas resolve o atraso?

A) st.rerun() depois de atualizar

B) Apagar o placar

C) Usar st.write

D) Recarregar a página à mão

A. Ou usar on_click no botão.

3. Por que o teste confere também os st.metric?

A) Por enfeite

B) O estado pode estar certo e a tela atrasada

C) O AppTest exige

D) Para testar as imagens

B. O usuário vê a tela.

4. Como o "Zerar placar" funciona no código de referência?

A) Fecha o app

B) Apaga as chaves da sessão; o placar é recriado com zeros

C) Muda o jogo.py

D) Recarrega o navegador

B. Com st.rerun() em seguida.

Placar certo, *na hora certa*

O jogo ganhou **placar** com `st.metric`, **histórico** num `st.expander` e **zerar**. A lição do Streamlit: o script roda de cima para baixo, então quem muda um valor já desenhado chama `st.rerun()` (ou usa `on_click`). E o teste confere a **tela**, não só o estado.

`st.metric`

Histórico

`st.expander`

`st.rerun()`

Testar a tela

Próxima aula → 16 · Projeto Analisador de Texto · Primeira Versão

Fontes: cursor.com/docs (Agent Overview, Debug Mode, Browser); docs.streamlit.io (`st.metric`, `st.expander`, Session State, Button behavior and callbacks, `st.rerun`, `AppTest`), consultados em outubro de 2026. Código de referência em `projeto/`, testado com Streamlit 1.65.